REDIS ROADMAP · PHASE 7

# Production Redis

*From "it works on my machine" to reliable at scale*

An end-to-end, self-contained handbook. Read it top to bottom and you'll understand how to run Redis in production: persistence, replication, automatic failover, sharding, monitoring, and disaster recovery — every concept backed by a demo you can actually run.

### What's inside


1. Why production Redis is a different game
2. Concept 1 — Persistence (RDB + AOF)
3. Concept 2 — Replication (master & replicas)
4. Concept 3 — Sentinel (automatic failover / HA)
5. Concept 4 — Cluster (horizontal sharding)
6. Concept 5 — Monitoring & Observability
7. Concept 6 — Disaster Recovery
8. Deploying it & the production checklist


Prerequisite: Phases 1–6 complete · Every demo runs real Redis instances locally

CHAPTER 1

# Why Production Redis Is Different

Everything you built in Phases 1–6 ran on a single Redis on `localhost`. That's perfect for learning, but in production a single Redis is a ticking clock: RAM gets wiped on restart, one machine is a single point of failure, one machine has a memory ceiling, and problems are invisible until users complain. This phase closes all of those gaps.

Six capabilities turn a dev toy into a production system:

| Concept | Protects you from | Redis mechanism |
|---|---|---|
| **Persistence** | Losing data on restart | RDB snapshots + AOF log |
| **Replication** | Read overload; node loss | Master → replicas |
| **Sentinel** | Master death (downtime) | Automatic failover |
| **Cluster** | Hitting one machine's limits | Sharding across masters |
| **Monitoring** | Silent degradation | INFO / SLOWLOG / exporters |
| **Disaster recovery** | Losing everything at once | Off-site RDB backups |

> **HOW TO READ THIS HANDBOOK**
>
> Each concept has a matching script in `concepts/` that spins up *real* Redis instances on your machine — starting masters, replicas, sentinels, and even a live cluster — so you can watch failover and sharding happen for yourself, not just read about them.

CHAPTER 2 · CONCEPT 1

# Persistence (RDB + AOF)

file `concepts/01_persistence.py`

Redis lives in RAM, which is wiped on restart. Persistence writes data to disk so it survives. Redis offers two mechanisms — learn both, then use both.

| | RDB (snapshots) | AOF (write log) |
|---|---|---|
| What | A photo of the whole dataset, taken periodically | A recording of every write command |
| Analogy | Camera photo every few minutes | Security-camera video |
| Pros | Tiny file, fast restart, ideal for backups | Minimal data loss (fsync every 1s) |
| Cons | Lose writes since the last snapshot | Bigger file, slightly slower |

```
# The production answer: enable BOTH
appendonly yes            # AOF for durability (lose <= ~1s)
appendfsync everysec      # flush the log once per second
save 900 1                # RDB snapshot as a compact backup file
save 300 10
```

**THE DEMO PROVES IT**

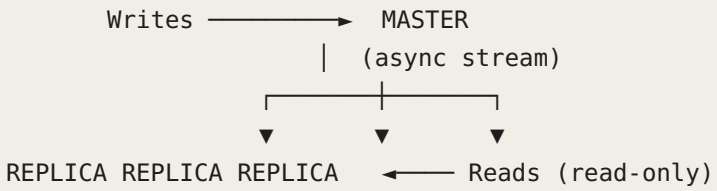
The script writes a session, a checkpoint, and a queue; kills the Redis process; restarts it from the same data dir; and shows every value came back. Without persistence, all of it would return `None`.

CHAPTER 3 · CONCEPT 2

# Replication

file `concepts/02_replication.py`

A replica holds a live copy of the master's data. This buys you two things: **read scaling** (send reads to replicas) and **redundancy** (a replica can be promoted if the master dies).




A replica is created with one directive — `replicaof host port` — then it does an initial full sync and receives every later write as a stream.

> **THE ONE GOTCHA: ASYNC = EVENTUAL CONSISTENCY**
>
> Replication is asynchronous, so a write reaches replicas a moment later (usually sub-millisecond). A replica read can therefore be a hair stale. Fine for caches; be careful when you need read-your-own-writes. The demo also shows replicas reject writes — reads to replicas, writes to the master.

CHAPTER 4 · CONCEPT 3

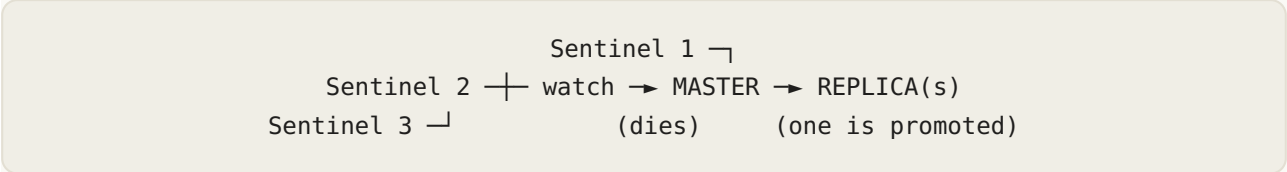
# Sentinel — Automatic Failover

file `concepts/03_sentinel.py`

Replication gives you copies, but if the master dies, *who* promotes a replica and tells your app the new address? Doing that by hand at 3am is how outages become disasters. **Sentinel** automates it.

Sentinel is a separate process that watches your master and, on failure, automatically:

1. detects the master is down,
2. elects a replica and promotes it to master,
3. reconfigures the other replicas to follow the new master,
4. tells clients the new master's address.




You run an **odd number** of sentinels (usually 3) so they can vote, and require a **quorum** (e.g. 2 of 3) to agree the master is down before failing over — this stops one confused sentinel from causing chaos. Clients don't hard-code the master; they ask a sentinel:

```
SENTINEL get-master-addr-by-name mymaster   # returns the CURRENT master
```

> **THE DEMO ACTUALLY FAILS OVER**
>
> It starts a master, a replica, and 3 sentinels, then *kills the master* and polls a sentinel until it reports a new master — typically in a few seconds. The promoted replica keeps the data and starts accepting writes, **with zero manual action**.

CHAPTER 5 · CONCEPT 4

# Cluster — Horizontal Sharding

file `concepts/04_cluster.py`

Replication scales reads, but every node still holds the whole dataset and all writes hit one master. When your data outgrows one machine's RAM, or one master can't take the write load, you **shard** with Redis Cluster: the data is split across multiple masters.

## Hash slots

The keyspace is divided into **16384 hash slots**. Each key maps to a slot via `CRC16(key) % 16384`, and each master owns a range of slots — so every key belongs to exactly one shard.

```
Master A: slots     0 – 5460
Master B: slots  5461 – 10922
Master C: slots 10923 – 16383
```

Ask the wrong node and it replies `MOVED` with the right node's address; cluster-aware clients cache this map and talk straight to the owner.

> **HASH TAGS: KEEP RELATED KEYS TOGETHER**
>
> Multi-key commands only work if the keys share a shard. Wrap the common part in braces so only that part is hashed: `cart:{user42}:items` and `cart:{user42}:total` land on the same shard. The demo builds a real 3-master + 3-replica cluster and shows 12 keys distributing across the masters.

CHAPTER 6 · CONCEPT 5

# Monitoring & Observability

file `concepts/05_monitoring.py`

You can't operate what you can't see. Redis exposes everything you need through `INFO`, `SLOWLOG`, and a few other commands. These are the signals to watch:

| Metric | Why it matters |
|---|---|
| Memory used vs maxmemory | #1 thing to watch — hitting the cap triggers eviction/rejection |
| Hit rate (hits ÷ hits+misses) | A low rate means the cache isn't earning its keep |
| Ops/sec | Your live load |
| Connected clients | Leaks and spikes show up here |
| Evicted keys | Climbing = out of memory for your workload |
| Slow log | Catches O(N) commands (KEYS, big SORT) that block Redis |

**FROM SCRIPT TO DASHBOARD**

The demo computes a health report and fires alerts from thresholds — the same logic real systems run. In production you export these to Prometheus (via `redis_exporter`) and graph them in Grafana. A ready stack is in `docker/docker-compose.monitoring.yml`.

CHAPTER 7 · CONCEPT 6

# Disaster Recovery

file `concepts/06_disaster_recovery.py`

Persistence survives a restart; Sentinel survives one node dying. But some disasters take out everything at once: a region outage, a bad deploy running `FLUSHALL`, a corrupted volume. For those you need **off-site backups** and a **tested restore**.

## The backup is just the RDB file

1. Trigger a fresh snapshot with `BGSAVE`.
2. Copy `dump.rdb` to storage in another region (S3/GCS), on a schedule.
3. Keep several generations so you can roll back past a bad write.

## The restore is dropping the file in and starting

Place a backed-up `dump.rdb` in a fresh Redis's data dir and start it — Redis loads it on boot.

> **THE GOLDEN RULE**
>
> A backup you have never restored is not a backup — it's a hope. The demo fills a Redis, backs it up off-site, *destroys the primary and its data dir*, then restores into a brand-new Redis and verifies all 501 keys came back. The `scripts/` folder has cron-ready `backup.sh` and `restore.sh`.

CHAPTER 8

# Deploying It & The Checklist

## What's in this package beyond the demos

| Folder | Contents |
|---|---|
| `concepts/` | The 6 runnable demos (each spins up real Redis) |
| `configs/` | Commented `redis-production.conf`, `redis-replica.conf`, `sentinel.conf` |
| `docker/` | Compose files: replication, sentinel, cluster, and a Prometheus+Grafana monitoring stack |
| `scripts/` | `backup.sh`, `restore.sh`, `healthcheck.sh` |

## Which topology do I actually need?

| Situation | Use |
|---|---|
| Small app, data fits in RAM, some downtime OK | Single node + persistence + backups |
| Read-heavy, want redundancy | Replication (+ Sentinel for auto-failover) |
| Need zero-touch failover / HA | Sentinel (3 sentinels, quorum 2) |
| Data or writes exceed one machine | Cluster (shard across masters, each with a replica) |

## The production checklist

- **Persistence on** (AOF everysec + RDB), and **backups off-site** on a schedule.
- **Set `maxmemory` + an eviction policy** so Redis never eats the whole box.
- **Require a password** (or ACLs) and **disable dangerous commands** (FLUSHALL, KEYS).
- **Never run `KEYS` in production** — use `SCAN`; watch the slow log for O(N) commands.
- **Run 3+ Sentinels** on separate hosts with a quorum, if you need HA.
- **Export metrics** to Prometheus/Grafana and alert on memory %, hit rate, and evictions.
- **Practice the restore** regularly — an untested backup is a hope.

### The five-second summary of Phase 7

Persistence (RDB + AOF) keeps data across restarts. Replication scales reads and adds redundancy. Sentinel turns that redundancy into automatic failover. Cluster shards data across many masters when one machine isn't enough. Monitoring makes problems visible before users notice, and disaster recovery — tested, off-site backups — is your last line of defense. Together they take everything you built in Phases 1–6 and make it something you can trust in production.

## The roadmap so far, and what's next

You've gone from *caching fundamentals* (Phase 1) all the way to *running Redis reliably at scale* (Phase 7): SWE use cases, system design, AI/ML caching, agentic memory, and now production operations. The final step is the **capstone AI Research Assistant** that combines everything — LLM cache, embedding cache, session store, rate limiter, agent memory, checkpointing, pub/sub, and queues — deployed behind a load balancer with the HA and monitoring you just learned. You now have every building block it needs.